

CADDack: GNN Mathematics Reference

Two-Tower Bayesian Fusion Model for Protein–Ligand Binding Affinity

CADDack Project

June 25, 2026

Abstract

This document gives a self-contained mathematical description of every computation performed by the CADDack graph neural network stack, from raw molecular input to predicted binding affinity with calibrated uncertainty. The architecture consists of (1) a 2D ligand message-passing encoder (GCN / GINE), (2) a 3D geometry tower (SchNet-style continuous-filter convolution), (3) a Bayesian fusion head (Bayes-by-Backprop, heteroscedastic ELBO), and (4) Monte Carlo uncertainty decomposition at inference. Each formula is cross-referenced to its implementing source file.

Contents

1	Input Representation	3
1.1	Atom feature vector (<code>datasets.py:atom_features</code>)	3
1.2	Bond feature vector (<code>datasets.py:bond_features</code>)	3
2	Tower 1 — Ligand 2D Message Passing	3
2.1	MolecularGCN (<code>models.py:MolecularGCN</code>)	3
2.2	MolecularGINE (<code>models.py:MolecularGINE</code>)	4
3	Tower 2 — 3D Geometry Tower	4
3.1	Atom embedding	4
3.2	Radius graph	4
3.3	Gaussian radial basis functions (<code>models.py:GaussianRBF</code>)	5
3.4	Cosine envelope	5
3.5	Interaction block (<code>models.py:InteractionBlock</code>)	5
3.6	Readout	6
3.7	SE(3) invariance	6
4	Bayesian Fusion Head	6
4.1	Fusion	6
4.2	Variational posterior (<code>bayes.py:BayesianLinear</code>)	6
4.3	Reparameterisation trick	7
4.4	KL divergence (<code>bayes.py:BayesianLinear.kl</code>)	7
4.5	BayesianMLP output heads (<code>bayes.py:BayesianMLP</code>)	7
4.6	Heteroscedastic Gaussian NLL	7
4.7	ELBO training objective (<code>bayes.py:elbo_loss</code>)	8
4.8	KL warmup schedule (<code>train.py:train_fusion_from_complexes</code>)	8
5	Uncertainty Decomposition at Inference	8

6	Training Losses for Standalone Models	9
7	Architecture Summary	10

§ 1 Input Representation

1.1 Atom feature vector (`datasets.py:_atom_features`)

Each heavy atom i in a molecule is represented by a seven-dimensional feature vector $\mathbf{x}_i \in \mathbb{R}^7$:

$$\mathbf{x}_i = [z_i, \deg(i), q_i, \mathbf{1}_{\text{arom}}, \text{hyb}_i, n_i^H, \mathbf{1}_{\text{ring}}]^\top \quad (1)$$

where:

- $z_i \in \mathbb{Z}_{>0}$ is the atomic number (e.g. $z = 6$ for carbon, $z = 7$ for nitrogen).
- $\deg(i)$ is the number of explicit bonds (degree in the molecular graph).
- q_i is the formal charge.
- $\mathbf{1}_{\text{arom}} \in \{0, 1\}$ indicates aromaticity.
- $\text{hyb}_i \in \{1, \dots, 6\}$ is the integer encoding of the hybridisation state (SP, SP2, SP3, ...).
- n_i^H is the total hydrogen count (implicit + explicit).
- $\mathbf{1}_{\text{ring}} \in \{0, 1\}$ indicates ring membership.

All entries are cast to `float32`.

1.2 Bond feature vector (`datasets.py:_bond_features`)

Each bond (i, j) in the molecular graph is represented by a seven-dimensional edge feature vector $\mathbf{e}_{ij} \in \mathbb{R}^7$:

$$\mathbf{e}_{ij} = [\mathbf{1}_{\text{single}}, \mathbf{1}_{\text{double}}, \mathbf{1}_{\text{triple}}, \mathbf{1}_{\text{arom}}, \mathbf{1}_{\text{conj}}, \mathbf{1}_{\text{ring}}, \text{stereo}]^\top \quad (2)$$

The first four elements form a one-hot encoding of bond order/type. $\text{stereo} \in \{0, 1, 2, \dots\}$ is the integer cast of the RDKit stereo enum.

Edges are **bidirectional**: for each bond (i, j) both directed edges $(i \rightarrow j)$ and $(j \rightarrow i)$ are inserted with identical features.

§ 2 Tower 1 — Ligand 2D Message Passing

2.1 MolecularGCN (`models.py:MolecularGCN`)

The Graph Convolutional Network [1] applies the symmetric-normalised spectral convolution at each layer ℓ :

$$\mathbf{x}_i^{(\ell+1)} = \text{ReLU} \left(W^{(\ell)} \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{\mathbf{x}_j^{(\ell)}}{\sqrt{d_i d_j}} \right) \quad (3)$$

where $d_i = \deg(i) + 1$ (self-loop included) and $W^{(\ell)} \in \mathbb{R}^{C \times C}$ is the learnable weight matrix. Dropout with rate p is applied after each non-linearity.

After L convolutional layers, global *mean* pooling produces the molecular embedding:

$$\mathbf{h}_{\text{mol}} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbf{x}_i^{(L)} \in \mathbb{R}^C \quad (4)$$

The prediction head is a single affine map: $\hat{y} = \mathbf{w}^\top \mathbf{h}_{\text{mol}} + b$.

2.2 MolecularGINE (models.py:MolecularGINE)

The Graph Isomorphism Network with Edge features (GINE) [2, 3] extends GIN [2] to incorporate bond features. At layer ℓ :

$$\mathbf{x}_i^{(\ell+1)} = \text{MLP}^{(\ell)} \left((1 + \varepsilon) \mathbf{x}_i^{(\ell)} + \sum_{j \in \mathcal{N}(i)} \text{ReLU}(\mathbf{x}_j^{(\ell)} + \mathbf{e}_{ij}) \right) \quad (5)$$

Key design choices:

- The edge features $\mathbf{e}_{ij}$ are added (not concatenated) to the sender embedding before aggregation. This is algebraically equivalent to a first-order Taylor expansion of an edge-conditioned filter.
- Aggregation is **summation**, which is strictly more expressive than mean or max and makes GINE at least as powerful as the Weisfeiler–Leman (1-WL) graph isomorphism test [2].
- Each $\text{MLP}^{(\ell)}$ is a two-layer network: $\text{Linear}(C_{\text{in}} \rightarrow C) \rightarrow \text{ReLU} \rightarrow \text{Linear}(C \rightarrow C)$.

After L layers, global *add* pooling:

$$\mathbf{h}_{\text{lig}} = \sum_{i \in \mathcal{V}} \mathbf{x}_i^{(L)} \in \mathbb{R}^C \quad (6)$$

GINE is the recommended ligand encoder because bond-order information (double bonds, aromaticity) is essential for molecular property prediction.

§ 3 Tower 2 — 3D Geometry Tower

Based on SchNet [4]. The tower operates exclusively on *interatomic distances*, which guarantees SE(3) invariance.

3.1 Atom embedding

$$\mathbf{h}_i^{(0)} = \text{Embedding}(z_i) \in \mathbb{R}^C \quad (7)$$

A learned look-up table maps each integer atomic number z_i to a C -dimensional vector.

3.2 Radius graph

Edges are created between all atom pairs within cutoff c (default $c = 6.0 \text{ \AA}$):

$$\mathcal{E} = \{(i, j) : \|\mathbf{r}_i - \mathbf{r}_j\|_2 \leq c, i \neq j\} \quad (8)$$

The edge set spans ligand–ligand, pocket–pocket, *and* ligand–pocket pairs. The ligand–pocket cross-edges represent the direct binding contacts that the model must learn from. The radius

graph is computed *inside* `forward()` using the per-batch position tensors so that no pre-computed `edge_index` ever needs PyG collation across differently-sized graphs.

3.3 Gaussian radial basis functions (`models.py:GaussianRBF`)

The scalar distance $d_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|_2$ is expanded into K basis functions:

$$\phi_k(d) = \exp\left(-\frac{(d - \mu_k)^2}{\gamma}\right), \quad k = 1, \dots, K \quad (9)$$

where the centres μ_k are equally spaced on $[0, c]$:

$$\mu_k = \frac{(k-1)c}{K-1}, \quad \gamma = \left(\frac{c}{K}\right)^2 \quad (10)$$

The resulting feature vector $\phi(d_{ij}) \in \mathbb{R}^K$ is a soft histogram over distance ranges ($K = 50$ by default).

3.4 Cosine envelope

To ensure that messages vanish smoothly at the cutoff boundary (preventing a discontinuity in the learned function at $d = c$):

$$w(d) = \frac{1}{2} \left(\cos\left(\frac{\pi d}{c}\right) + 1 \right) \quad (11)$$

$$w(0) = 1, \quad w(c/2) = 1/2, \quad w(c) = 0.$$

3.5 Interaction block (`models.py:InteractionBlock`)

Each of the T interaction blocks ($T = 3$ by default) performs the following steps.

Filter generation.

$$\mathbf{f}_{ij}^{(\ell)} = \text{FilterNet}^{(\ell)}(\phi(d_{ij})) \cdot w(d_{ij}) \in \mathbb{R}^C \quad (12)$$

where `FilterNet` is a two-layer MLP with SiLU activations:

$$\text{FilterNet}(\mathbf{u}) = W_2 \text{SiLU}(W_1 \mathbf{u} + b_1) + b_2, \quad \text{SiLU}(x) = x \sigma(x) \quad (13)$$

Continuous-filter message.

$$\mathbf{m}_{ij}^{(\ell)} = \mathbf{h}_j^{(\ell)} \odot \mathbf{f}_{ij}^{(\ell)} \in \mathbb{R}^C \quad (14)$$

The sender embedding is modulated element-wise by the distance-dependent filter. Unlike graph convolutions with fixed weight matrices, the filter here is a *continuous function of distance* that is learned end-to-end.

Aggregation and residual update.

$$\mathbf{a}_i^{(\ell)} = \sum_{j: (i,j) \in \mathcal{E}} \mathbf{m}_{ij}^{(\ell)} \quad (15)$$

$$\mathbf{h}_i^{(\ell+1)} = \mathbf{h}_i^{(\ell)} + \text{UpdateMLP}^{(\ell)}(\mathbf{a}_i^{(\ell)}) \quad (16)$$

UpdateMLP: $\text{Linear}(C \rightarrow C) \rightarrow \text{SiLU} \rightarrow \text{Linear}(C \rightarrow C)$. The residual connection stabilises training.

3.6 Readout

$$\mathbf{h}_{\text{geo}} = \sum_{i \in \mathcal{V}} \mathbf{h}_i^{(T)} \in \mathbb{R}^C \quad (17)$$

3.7 SE(3) invariance

The geometry tower is invariant to all rigid-body transformations of the complex. For any rotation $R \in \text{SO}(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$:

$$\|(R\mathbf{r}_i + \mathbf{t}) - (R\mathbf{r}_j + \mathbf{t})\|_2 = \|\mathbf{r}_i - \mathbf{r}_j\|_2 \quad (18)$$

Since every computation depends only on d_{ij} , the output is unchanged under rotation or translation of the entire complex.

§ 4 Bayesian Fusion Head

Based on Bayes-by-Backprop [5]. Towers are deterministic; only the fusion head is Bayesian. This keeps the KL tractable and training stable.

4.1 Fusion

The two tower outputs are concatenated:

$$\mathbf{h} = [\mathbf{h}_{\text{lig}}; \mathbf{h}_{\text{geo}}] \in \mathbb{R}^{2C} \quad (19)$$

4.2 Variational posterior (bayes.py:BayesianLinear)

Each weight w_{ij} in the fusion MLP has an independent Gaussian posterior:

$$q(w_{ij}) = \mathcal{N}(\mu_{ij}, \sigma_{ij}^2) \quad (20)$$

The standard deviation is parameterised via the softplus function to ensure positivity:

$$\sigma_{ij} = \text{softplus}(\rho_{ij}) = \ln(1 + e^{\rho_{ij}}) \quad (21)$$

Initialisation: μ_{ij} with Kaiming uniform; $\rho_{ij} = -5$, so

$$\sigma_{ij}|_{\rho=-5} = \text{softplus}(-5) = \ln(1 + e^{-5}) \approx 0.0067 \quad (22)$$

The network starts near-deterministic, giving the mean parameters time to reach useful values before the posterior widens.

4.3 Reparameterisation trick

To allow gradient-based optimisation through stochastic weights:

$$w_{ij} = \mu_{ij} + \sigma_{ij} \varepsilon_{ij}, \quad \varepsilon_{ij} \sim \mathcal{N}(0, 1) \quad (23)$$

The randomness is moved into ε_{ij} , which carries no parameters. Gradients flow through μ_{ij} and ρ_{ij} :

$$\frac{\partial w_{ij}}{\partial \mu_{ij}} = 1, \quad \frac{\partial w_{ij}}{\partial \rho_{ij}} = \varepsilon_{ij} \sigma'(\rho_{ij}) = \varepsilon_{ij} \sigma(\rho_{ij})(1 - \sigma(\rho_{ij})) \quad (24)$$

4.4 KL divergence (`bayes.py:BayesianLinear.kl`)

The prior is $p(w) = \mathcal{N}(0, \sigma_{\text{prior}}^2)$. The closed-form KL between posterior and prior is:

$$\text{KL}(q \| p) = \sum_{i,j} \left[\ln \frac{\sigma_{\text{prior}}}{\sigma_{ij}} + \frac{\sigma_{ij}^2 + \mu_{ij}^2}{2 \sigma_{\text{prior}}^2} - \frac{1}{2} \right] \quad (25)$$

This equals zero when $\mu_{ij} = 0$ and $\sigma_{ij} = \sigma_{\text{prior}}$ (posterior equals prior). As the network learns, μ_{ij} shifts away from zero and σ_{ij} compresses, increasing the KL. The term acts as a regulariser on the magnitude and certainty of learned weights.

The total KL over the entire Bayesian MLP is the sum over all layers:

$$\text{KL}_{\text{total}} = \sum_{d=1}^D \text{KL}_d \quad (26)$$

4.5 BayesianMLP output heads (`bayes.py:BayesianMLP`)

After D Bayesian hidden layers with ReLU activations, two *deterministic* linear output heads produce:

$$\hat{\mu} = \mathbf{w}_{\mu}^{\top} \mathbf{x}^{(D)} + b_{\mu} \in \mathbb{R} \quad (\text{predicted affinity mean}) \quad (27)$$

$$s = \text{clamp}(\mathbf{w}_s^{\top} \mathbf{x}^{(D)} + b_s, -10, 10) \in \mathbb{R} \quad (\text{log predictive variance, } s = \log \sigma_{\text{al}}^2) \quad (28)$$

The clamp prevents numerical overflow in e^s and e^{-s} .

4.6 Heteroscedastic Gaussian NLL

The model treats the affinity target as a Gaussian: $y \sim \mathcal{N}(\hat{\mu}, e^s)$. The per-example negative log-likelihood (dropping the additive constant $\frac{1}{2} \ln 2\pi$) is:

$$\ell_i = \frac{1}{2} \left[e^{-s_i} (y_i - \hat{\mu}_i)^2 + s_i \right] \quad (29)$$

The term e^{-s_i} reweights the squared error by the *inverse predicted variance*: when the model signals high aleatoric uncertainty (large s_i), the squared error is discounted and the model only pays the entropy penalty s_i . This makes the loss *self-calibrating* — the model is penalised for being overconfident and rewarded for expressing appropriate uncertainty.

The batch NLL is the mean:

$$\text{NLL}_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B \ell_i \quad (30)$$

4.7 ELBO training objective (`bayes.py:elbo_loss`)

Variational inference maximises the Evidence Lower Bound [5]:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{w \sim q} [\ln p(\mathbf{Y} \mid \mathbf{X}, w)] - \text{KL}(q(w) \parallel p(w)) \quad (31)$$

Minimising the negative ELBO per gradient step gives:

$$\mathcal{L} = \underbrace{\text{NLL}_{\text{batch}}}_{\text{reconstruction}} + \beta \cdot \underbrace{\frac{\text{KL}_{\text{total}}}{N_{\text{train}}}}_{\text{regularisation}} \quad (32)$$

where:

- The NLL is estimated with *one* weight sample per step (unbiased MC estimator of $\mathbb{E}_q[\text{NLL}]$).
- N_{train} normalises the KL to per-example scale, preventing the regularisation term from dominating as the dataset grows.
- β is the annealed KL weight (Section 4.8).

4.8 KL warmup schedule (`train.py:train_fusion_from_complexes`)

$$\beta(\text{epoch}) = \beta_{\text{max}} \cdot \min\left(1, \frac{\text{epoch} + 1}{\tau}\right) \quad (33)$$

where $\beta_{\text{max}} = \text{kl_weight}$ (default 1.0) and $\tau = \text{kl_warmup}$ (default 10 epochs).

Why warmup? At epoch 0, a full KL penalty forces $\sigma_{ij} \rightarrow \sigma_{\text{prior}}$ (posterior collapses to the prior mean). The means μ_{ij} receive no useful gradient, and the network never escapes the prior. By starting at $\beta = 0$ and ramping linearly, the network first builds informative mean parameters before the regulariser becomes active.

§ 5 Uncertainty Decomposition at Inference

At test time, T independent forward passes are drawn by sampling different weight realisations:

$$\{(\hat{\mu}_t, s_t)\}_{t=1}^T, \quad \hat{\mu}_t = \hat{\mu}(w_t), \quad w_t \sim q(w) \quad (34)$$

Predictive mean.

$$\bar{y} = \frac{1}{T} \sum_{t=1}^T \hat{\mu}_t \quad (35)$$

Epistemic uncertainty. Model uncertainty, reducible with more data:

$$\sigma_{\text{ep}}^2 = \text{Var}_t(\hat{\mu}_t) = \frac{1}{T} \sum_{t=1}^T (\hat{\mu}_t - \bar{y})^2 \quad (36)$$

Aleatoric uncertainty. Irreducible noise intrinsic to the data:

$$\sigma_{\text{al}}^2 = \frac{1}{T} \sum_{t=1}^T e^{s_t} = \mathbb{E}_t[e^{s_t}] \quad (37)$$

Total predictive variance. By the law of total variance:

$$\sigma_{\text{total}}^2 = \underbrace{\text{Var}_t(\hat{\mu}_t)}_{\sigma_{\text{ep}}^2} + \underbrace{\mathbb{E}_t[\sigma_{\text{al},t}^2]}_{\sigma_{\text{al}}^2} = \text{Var}[Y] = \text{Var}[\mathbb{E}[Y|W]] + \mathbb{E}[\text{Var}[Y|W]] \quad (38)$$

Both standard deviations (σ_{ep} , σ_{al}) are returned in the same units as the affinity target (e.g. pK_d units).

§ 6 Training Losses for Standalone Models

The standalone GCN and GINE models (no Bayesian head) are trained with:

Task	Loss	Implemented in
Classification	$\mathcal{L}_{\text{BCE}} = -\frac{1}{B} \sum_i [y_i \ln \sigma(\hat{y}_i) + (1 - y_i) \ln(1 - \sigma(\hat{y}_i))]$	<code>train.py:step</code>
Regression	$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_i (\hat{y}_i - y_i)^2$	<code>train.py:step</code>

Evaluation metrics:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (39)$$

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (40)$$

$$\text{AUC-ROC} = \Pr(\hat{p}_{\text{pos}} > \hat{p}_{\text{neg}}) \quad (41)$$

§ 7 Architecture Summary

Component	Input	Output	Default dim	Source
Atom embedding (Tower 2)	$z_i \in \mathbb{Z}$	$\mathbf{h}_i^{(0)} \in \mathbb{R}^C$	$C = 128$	<code>models.py</code>
GINE layers (Tower 1)	$\mathbf{x}_i^{(0)} \in \mathbb{R}^7$	$\mathbf{h}_{\text{lig}} \in \mathbb{R}^C$	$L = 3$	<code>models.py</code>
GaussianRBF	$d_{ij} \in \mathbb{R}$	$\phi(d_{ij}) \in \mathbb{R}^K$	$K = 50$	<code>models.py</code>
InteractionBlock ($\times T$)	$\mathbf{h}^{(\ell)}, \mathcal{E}, \phi$	$\mathbf{h}^{(\ell+1)} \in \mathbb{R}^C$	$T = 3$	<code>models.py</code>
Geometry readout	$\mathbf{h}_i^{(T)}$	$\mathbf{h}_{\text{geo}} \in \mathbb{R}^C$	$C = 128$	<code>models.py</code>
Fusion concat	$\mathbf{h}_{\text{lig}}, \mathbf{h}_{\text{geo}}$	$\mathbf{h} \in \mathbb{R}^{2C}$	$2C = 256$	<code>models.py</code>
BayesianLinear ($\times D$)	$\mathbf{h}$	$\mathbf{x}^{(D)} \in \mathbb{R}^{H_D}$	$[256, 128]$	<code>bayes.py</code>
Output heads	$\mathbf{x}^{(D)}$	$\hat{\mu}, s \in \mathbb{R}$	—	<code>bayes.py</code>
MC uncertainty ($\times T$)	T samples	$\bar{y}, \sigma_{\text{ep}}, \sigma_{\text{al}}$	$T = 30$	<code>models.py</code>

References

- [1] T. N. Kipf and M. Welling, *Semi-supervised classification with graph convolutional networks*, ICLR, 2017.
- [2] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, *How powerful are graph neural networks?*, ICLR, 2019.
- [3] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, *Strategies for pre-training graph neural networks*, ICLR, 2020.
- [4] K. T. Schütt, F. Arbabzadah, S. Chmiela, K. R. Müller, and A. Tkatchenko, *Quantum-chemical insights from deep tensor neural networks*, Nature Communications, 2017.
- [5] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, *Weight uncertainty in neural networks*, ICML, 2015.